

Cross Language Asymmetric Encryption

Using C# and Javascript

Introduction

Project Context

Developed a

PCI DSS v4.0 Compliant Secure Payment Card Gateway

for a major conglomerate

Technology Stack

- .NET Framework (Microsoft)
- Blazor WebAssembly (Client-side C# rendering)

The Challenge

Requirement:

Encrypt sensitive payment data (card number, CVV, expiry date) on the client side using RSA asymmetric encryption

The Problem

- C# cryptography libraries require operating system access
- OS-level cryptographic APIs are **not available in browser environments**
- Frontend encryption cannot use C#'s native crypto libraries

Solution: JavaScript interoperability with C# for client-side encryption

Key Challenges Faced

Algorithm Compatibility

Ensuring RSA implementation consistency between C# and JavaScript environments

Public Key Formatting

Converting and standardizing key formats across different cryptographic libraries

Data Format Consistency

Maintaining uniform data encoding and padding schemes between platforms

Dual API Integration

Coordinating between .NET and Web Crypto API specifications

RSA Asymmetric Encryption

RSA (Rivest-Shamir-Adleman) is a public-key cryptosystem that uses two different keys:
a public key for encryption and a private key for decryption

Types of Keys in Public Key Cryptography

Public Key

- Shared openly with anyone
- Used to **encrypt data**
- Cannot decrypt data
- Safe to distribute publicly

Private Key

- Kept secret and secure
- Used to **decrypt data**
- Mathematically linked to public key
- Must never be shared

Solution Architecture

JavaScript (Frontend)

1. Receive public key (XML format)
2. Convert XML → JWK format
3. Import key via Web Crypto API
4. Encrypt with RSA-OAEP-SHA256
5. Output Base64 encrypted string



C# (Backend)

1. Generate RSA key pair
2. Export public key as XML
3. Receive encrypted Base64
4. Decrypt with private key
5. Extract original JSON data

Key Format Conversion

Challenge: C# uses XML format, JavaScript Web Crypto API requires JWK format

C# XML Format

```
<RSAKeyValue>
  <Modulus>base64...</Modulus>
  <Exponent>AQAB</Exponent>
</RSAKeyValue>
```



JavaScript JWK Format

```
{
  "kty": "RSA",
  "n": "base64url...",
  "e": "AQAB"
}
```

Conversion Process

1. Parse XML to extract Modulus and Exponent 2. Decode Base64 to byte arrays 3. Convert to Base64URL encoding 4. Construct JWK object for Web Crypto API

C# Decryption

Using: System.Security.Cryptography (.NET Framework)

RSA Decryption Implementation

```
// Import private key
using (RSA rsa = RSA.Create())
{
    rsa.FromXmlString(privateKeyXml);

    // Decode Base64 to bytes
    byte[] encryptedBytes =
        Convert.FromBase64String(encryptedBase64);

    // Decrypt with RSA-OAEP-SHA256
    byte[] decryptedBytes = rsa.Decrypt(
        encryptedBytes,
        RSAEncryptionPadding.OaepSHA256
    );

    return Encoding.UTF8.GetString(decryptedBytes);
}
```


C# Decryption

Using: System.Security.Cryptography (.NET Framework)

RSA Decryption Implementation

```
// Import private key
using (RSA rsa = RSA.Create())
{
    rsa.FromXmlString(privateKeyXml);

    // Decode Base64 to bytes
    byte[] encryptedBytes =
        Convert.FromBase64String(encryptedBase64);

    // Decrypt with RSA-OAEP-SHA256
    byte[] decryptedBytes = rsa.Decrypt(
        encryptedBytes,
        RSAEncryptionPadding.OaepSHA256
    );

    return Encoding.UTF8.GetString(decryptedBytes);
}
```

Applications of RSA Encryption



Secure Online Transactions

E-commerce, banking, and payment gateways use RSA to protect credit card and financial data



Digital Signatures

Verify authenticity and integrity of documents, emails, and software updates



SSL/TLS Certificates

Secure HTTPS connections between web browsers and servers



Email Encryption

PGP and S/MIME use RSA to encrypt email messages and attachments

Questions & Answers

Thank You!



Asif Ahmed



❑ asifahmed3959.github.io

❑ linkedin.com/in/asifahmed3959

Scan QR codes to connect: